CERTIFICATE

CERTIFICATE OF PROJECT COMPLETION

This is to certify that the project entitled

"ACADRIX: STUDENT PERFORMANCE PREDICTION SYSTEM USING MACHINE LEARNING"

submitted by

Name: Mohammad Farhan, Tanish Yadav, Garima Jaiswal

of B-Tech (Artificial Intelligence & Machine Learning) has been successfully completed under supervision as part of the AIML Summer Internship 2026, IIHMF, Motilal Nehru National Institute of Technology (MNNIT) Allahabad, Prayagraj, during the academic session 2025–26.

The work presented in this project report is original and has been carried out by the students under guidance. To the best of my knowledge, the project has not been submitted elsewhere for the award of any degree or diploma.

Signature of Project Guide	Signature of Coordinator

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide for valuable guidance, encouragement and support throughout the development of Acadrix, our Student Performance Prediction System.

We are thankful to the organisers of the AIML Summer Internship 2026 at IIHMF and to the faculty members of Motilal Nehru National Institute of Technology (MNNIT) Allahabad for giving us the opportunity to apply machine learning to a real problem, and for the resources and knowledge they shared.

We would also like to thank our friends and classmates for their cooperation and constructive feedback during the various stages of the project, especially while testing the application.

Finally, we express our heartfelt gratitude to our parents and family members for their constant encouragement, motivation and support throughout our academic journey.

Student Names: Mohammad Farhan, Tanish Yadav, Garima Jaiswal

TABLE OF CONTENTS

Chapters	Title	Page No.
Certificate		**i**
Acknowledgement		**ii**
Table of Contents		**iii**
Abstract		**iv**
Chapter 1	**Introduction**	**1**
Chapter 2	**Literature Review**	**3**
Chapter 3	**Methodology**	**7**
Chapter 4	**Implementation**	**11**
Chapter 5	**Results and Discussion**	**18**
Chapter 6	**Conclusion and Future Scope**	**21**
References		**23**
Appendix		**24**

ABSTRACT

Acadrix: Student Performance Prediction System Using Machine Learning

Colleges usually discover that a student is struggling only when final results arrive, which is too late to help. The warning signs, such as falling attendance, missed assignments and weak internal marks, are recorded all semester but rarely acted upon in time.

This project presents Acadrix, a Student Performance Prediction System that predicts a student's final score (0–100) from five academic indicators: attendance, assignment score, internal assessment marks, previous semester score and weekly study hours. A dataset of 1,015 student records was cleaned, explored, and reduced through feature selection to the five features that matter.

Three regression algorithms, Linear Regression, Random Forest and XGBoost, were trained and compared on a held-out test set. Linear Regression performed best with an R^2 score of 0.901, a Mean Absolute Error (MAE) of 3.32 marks and a Root Mean Square Error (RMSE) of 4.17, and was deployed.

The model powers a complete web application with separate teacher and student portals, built with a Flask API, a SQLite database and a responsive frontend, and deployed online on Render. Teachers see at-risk students early; students see their predicted score and a concrete plan to reach a target score.

CHAPTER 1: INTRODUCTION

1.1 Background

Education plays a crucial role in the development of individuals and society, and academic performance is one of the most important indicators of a student's progress. Colleges collect a large amount of data about every student: attendance registers, assignment marks, internal tests and previous results. However, this data is mostly used after the fact, to report what already happened.

Educational Data Mining (EDM) is the field that applies data-mining and machine-learning methods to educational data to answer educational questions [1]. Predicting student performance is one of its most studied problems. The practical goal is early warning: if a model can estimate a student's final score part-way through the semester, teachers can offer extra classes, mentoring or counselling while there is still time to change the outcome. Reviews of the field consistently find that machine-learning predictions are most useful when they feed into such intervention plans [7].

Machine Learning (ML), a branch of Artificial Intelligence (AI), provides powerful techniques for identifying hidden patterns in data. Open-source libraries such as scikit-learn [10] and XGBoost [9] make strong models freely available, and a lightweight web application can put their predictions in front of teachers and students. Acadrix is built on this idea.

1.2 Problem Statement

Educational institutions often struggle to identify students who are at risk of poor performance before final examinations. Traditional evaluation relies on periodic tests and manual observation, and by the time results show a problem, the semester is over.

Problem: given a student's academic indicators part-way through a semester, predict their final score early enough for teachers to intervene and for students to change course.

This is a **regression** problem: the target is a continuous final score from 0 to 100. Each predicted score is then mapped to a grade (O, A+, A, B+, B, C, F) and to a risk level, as shown in Table 1.1.

Risk level	Predicted score	Meaning
Low	65 and above	On track
Medium	50 to 64.9	Needs some support
High	Below 50	Needs intervention now

Table 1.1: Risk levels used by the system

1.3 Objectives of the Project

Primary Objectives

- To predict a student's final score from attendance, assignment score, internal assessment, previous semester score and weekly study hours.
- To follow the complete machine learning lifecycle: data collection, preprocessing, EDA, feature engineering, model building, evaluation and deployment.
- To train and compare at least three regression models and deploy the best one.

Secondary Objectives

- To flag at-risk students early so teachers can act before examinations.
- To give each student a personal, actionable plan rather than a bare number.
- To show which academic factors influence the final score the most.

Operational Objectives

- To build a reproducible pipeline that can be retrained with one command.
- To deploy a secure web application that accepts user input and displays predictions.
- To keep student data private: each student sees only their own record.

1.4 Project Overview

The system follows this workflow:

1. Data Collection: generate a seeded dataset of 1,015 student records.
2. Data Cleaning: remove duplicates and impossible values, fill missing values, encode categories.
3. Exploratory Data Analysis: histograms, boxplots, scatter plots and a correlation heatmap.
4. Feature Engineering and Selection: test engineered features and keep only useful ones.
5. Model Development: train Linear Regression, Random Forest and XGBoost.
6. Model Evaluation: compare MAE, MSE, RMSE and R^2 on a held-out test set.
7. Deployment: serve the best model through a Flask web application on Render.

1.5 Chapter Summary

This chapter introduced student performance prediction, the problem of late identification of struggling students, and the objectives of the project. The next chapter reviews existing research in this area.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

Predicting student academic performance is an important research area within Educational Data Mining [1]. Researchers have applied statistical, data-mining and machine-learning methods to school and university data to identify at-risk students and improve learning outcomes. This chapter summarises the main approaches and key studies, and identifies the gap this project addresses.

2.2 Existing Approaches for Student Performance Prediction

Published work falls into four families of methods. Most studies frame the task as classification (pass/fail, grade band, at risk or not) rather than regression of an exact score [6].

2.2.1 Statistical Approaches

Linear and logistic regression relate a score or a pass/fail outcome to a weighted sum of inputs.

Advantages:

- Fast, and need little data.
- Easy to explain to teachers: each weight shows how much a factor matters.

Limitations:

- Assume the relationship is roughly linear.

2.2.2 Classical Machine Learning Approaches

Decision trees, Naive Bayes, k-nearest neighbours and support vector machines learn more flexible decision boundaries. Decision trees are especially common because their rules can be read directly [6].

Advantages:

- Capture non-linear patterns.
- Decision trees produce human-readable rules.

Limitations:

- Single trees can overfit the training data.

2.2.3 Ensemble Approaches

Random Forests [8] average many decision trees trained on random samples of rows and features. Gradient boosting methods such as XGBoost [9] build trees one after another, each correcting the errors of the previous ones.

Advantages:

- Usually the highest accuracy on tabular data.
- Provide feature-importance rankings.

Limitations:

- Slower to train and harder to explain than a single linear model.

2.2.4 Neural Network Approaches

Multi-layer neural networks, along with decision trees, are among the most frequently applied techniques in the field [6]. They can model complex patterns but need more data and are harder to interpret.

Across all of these methods, the most important inputs are consistent. **Prior academic achievement** (earlier grades, internal assessment, cumulative GPA) is the strongest and most used predictor [5][6]. **Class attendance** is also strong: a meta-analysis found it correlates with course grades at about r = 0.44, more strongly than other known predictors such as admission test scores, high-school GPA or study time [4].

2.3 Research Studies

Research Study 1: A Systematic Literature Review of Student Performance Prediction Using ML

Authors: Albreiki, Zaki and Alashwal (2021) [7]

Data and method: Systematic review of studies published from 2009 to 2021.

Findings: Machine-learning methods are effective at identifying students at risk and predicting dropout. Most studies use university databases or online learning platforms, and the authors stress that predictions must drive intervention plans.

Research Study 2: Predicting Academic Success in Higher Education

Authors: Alyahyan and Düştegör (2020) [6]

Data and method: Literature review and best-practice guide for higher education.

Findings: Prior academic achievement is the strongest factor in predicting success; decision trees and neural networks are the most frequently used techniques.

Research Study 3: A Review on Predicting Student Performance Using Data Mining

Authors: Shahiri, Husain and Rashid (2015) [5]

Data and method: Systematic review of data-mining prediction studies.

Findings: Internal assessment and CGPA are the most common attributes used to predict student performance.

Research Study 4: Class Attendance in College: A Meta-Analytic Review

Authors: Credé, Roch and Kieszczynka (2010) [4]

Data and method: Meta-analysis of 69 studies covering 21,195 students (course grades).

Findings: Attendance correlates with course grades (r = 0.44) and GPA (r = 0.41), more strongly than any other known predictor.

Research Study 5: Using Data Mining to Predict Secondary School Student Performance

Authors: Cortez and Silva (2008) [2]

Data and method: Portuguese secondary-school students (the UCI Student Performance data: 30 attributes, 649 students in the Portuguese-language set). Decision tree, Random Forest, neural network and SVM were compared.

Findings: Accuracy is high when earlier-period grades are available; predicting the final grade without them is much harder but more useful.

Research Study 6: Predicting Students' Performance in Distance Learning

Authors: Kotsiantis, Pierrakeas and Pintelas (2004) [3]

Data and method: A distance-learning university course; several ML classifiers.

Findings: An early study showing that machine learning can predict distance-learning student performance.

2.4 Comparative Analysis of Existing Studies

Study	Data	Method	Key finding
Albreiki et al. (2021)	Review, 2009–2021	ML for at-risk and dropout prediction	ML works; predictions should drive interventions
Alyahyan and Düştegör (2020)	Higher-education review	Review and best practices	Prior achievement is the strongest factor
Shahiri et al. (2015)	Review of prediction studies	Data-mining techniques	Internal assessment and CGPA most used
Credé et al. (2010)	69 studies, 21,195 students	Meta-analysis	Attendance r = 0.44 with grades
Cortez and Silva (2008)	649 secondary students	DT, RF, NN, SVM	Earlier grades make prediction easier
Kotsiantis et al. (2004)	Distance-learning course	Several ML classifiers	ML can predict distance-learning results

Table 2.1: Summary of the reviewed studies

2.5 Research Gap

Most published studies stop at a model and an accuracy figure, while reviews call for predictions to reach the people who can act on them [7]. Acadrix combines the two strongest predictor families from the literature, prior achievement (internal assessment and previous semester score) and attendance, with assignment scores and study hours. It predicts an exact score rather than only pass/fail, and delivers it through a working application: teachers get early-warning lists, and students get a personal plan.

2.6 Chapter Summary

This chapter reviewed statistical, classical machine-learning, ensemble and neural-network approaches, and six key studies. The literature agrees that prior achievement and attendance are the strongest predictors. The next chapter describes the methodology used in this project.

CHAPTER 3: METHODOLOGY

3.1 Introduction

The methodology defines the systematic approach used to build the prediction model, from raw data to a deployed application. Each step is implemented in Python scripts so the whole pipeline can be reproduced with two commands.

3.2 Dataset Description

The model is trained on a synthetic dataset generated by `ml/generate_dataset.py` with a fixed random seed (42), so every result in this report can be reproduced exactly. Each simulated student has two hidden traits, *ability* and *motivation*, that drive all their indicators, so the features are correlated the way real academic data is. The final score depends on the five academic features plus two real-world effects: diminishing returns on study hours, and an extra penalty for attendance below 75%.

Dataset Characteristics

- Dataset size: 1,015 student records, 12 columns (992 rows after cleaning).
- Data type: structured tabular data in CSV format.
- Target variable: final score (0–100), a continuous value.
- Dataset nature: numerical features, categorical features and a continuous output.
- Built-in quality problems: about 2% missing values, 15 duplicate rows and 8 impossible values.

Features in the Dataset

Column	Type	Range	Role
attendance_pct	Number	0–100 %	Candidate feature
assignment_score	Number	0–100	Candidate feature
internal_avg	Number	0–100	Candidate feature
prev_sem_score	Number	0–100	Candidate feature
study_hours_week	Number	0–30 h	Candidate feature
sleep_hours	Number	4–10 h	Candidate feature
commute_minutes	Number	5–90 min	Candidate feature
gender	Category	F / M	Candidate feature
extracurricular	Category	yes / no	Candidate feature
internet_access	Category	yes / no	Candidate feature
final_score	Number	0–100	Target variable

Table 3.1: Columns in the dataset

Example Dataset Records (raw file, before cleaning)

Attendance	Assignment	Internal	Prev. sem	Study h	Final
60.3	(missing)	44.4	60.4	5.3	50.5
58.8	58.0	79.9	73.7	6.7	66.0
96.4	87.0	77.3	78.3	15.0	78.6
83.6	64.0	59.3	84.5	7.0	69.5

Table 3.2: First four rows of data/students_dataset.csv

Why a synthetic dataset: the application's five inputs do not line up with the columns of common public datasets (for example, the UCI Student Performance data has no assignment score [2]). Any real dataset with the same column names can be used instead by re-running the training script.

3.3 Data Preprocessing

Raw data often contains duplicates, missing values and outliers. Cleaning took the raw file from 1,015 rows to **992 clean rows** in four steps:

1. **Remove duplicates:** 15 exact duplicate rows dropped (1,015 → 1,000).
2. **Treat outliers:** every column is checked against its valid range. 8 rows with impossible values (55–90 study hours a week, 130–160% attendance) were removed rather than clipped, because they are data-entry errors, not extreme students (1,000 → 992).
3. **Handle missing values:** 79 empty cells filled with the column median for numbers and the most common value for categories.
4. **Encode categories:** extracurricular and internet access mapped yes/no → 1/0; gender mapped to a 0/1 column.

Normalisation: Linear Regression uses a StandardScaler inside its pipeline, fitted on training data only. The tree models (Random Forest, XGBoost) do not need scaling.

3.4 Machine Learning Algorithms Used

Three regression algorithms were compared, one from each main family identified in Chapter 2.

3.4.1 Linear Regression

Linear Regression predicts the score as a weighted sum of the inputs, choosing the weights that minimise the squared error on the training data:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_5 x_5$$

Advantages:

- Fast, stable and easy to interpret.
- Each weight shows how much a factor moves the score.

Limitations:

- Cannot follow curved relationships.

3.4.2 Random Forest Regressor

Random Forest [8] trains B decision trees, each on a random sample of rows and a random subset of features at each split, and averages their predictions:

$$\hat{y} = (1/B)\, \Sigma\, T_{\beta}(x)$$

Advantages:

- Captures non-linear patterns.
- Resistant to overfitting.
- Gives feature-importance rankings.

Limitations:

- Slower and less interpretable than a linear model.

3.4.3 XGBoost Regressor

XGBoost [9] builds trees one after another; each new tree is fitted to the errors the earlier trees still make. It minimises a loss plus a penalty on tree complexity, which limits overfitting:

$$\hat{y} = \Sigma_k\, f_k(x), \quad L = \Sigma_i\, (y_i - \hat{y}_i)^2 + \Sigma_k\, (\gamma T_k + \tfrac{1}{2}\lambda \|w_k\|^2)$$

Advantages:

- Often the most accurate model on tabular data.
- Supports monotone constraints that keep predictions sensible.

Limitations:

- Many hyperparameters; slower to train.

3.5 Model Evaluation Metrics

For n test students with actual score y_i and predicted score $\hat{y}_i$:

Mean Absolute Error (MAE)

The average size of a prediction error, in marks. Lower is better.

$$MAE = (1/n)\,\Sigma\,|y_i - \hat{y}_i|$$

Mean Squared Error (MSE) and Root Mean Square Error (RMSE)

MSE squares each error, so large mistakes count more; RMSE brings it back to marks. Lower is better.

$$MSE = (1/n)\,\Sigma\,(y_i - \hat{y}_i)^2, \quad RMSE = \sqrt{MSE}$$

R-Squared Score (R^2)

The share of the variation in scores that the model explains: 1 is perfect, 0 is no better than predicting the average. Higher is better.

$$R^2 = 1 - \Sigma\,(y_i - \hat{y}_i)^2 / \Sigma\,(y_i - \bar{y})^2$$

3.6 Validation Strategy and Model Selection

The clean data is split once into 80% for training (793 students) and 20% for testing (199 students) with a fixed random seed. The test set is used only for the final comparison. To check that results do not depend on one lucky split, each model is also scored with 5-fold cross-validation on the training set.

The best model is selected by the highest R^2 on the test set, checked against RMSE, MAE and cross-validated R^2. Ties are broken in favour of the simpler, more explainable model.

3.7 Chapter Summary

This chapter described the dataset, the preprocessing steps, the three algorithms, the evaluation metrics and the validation strategy. The next chapter presents how these were implemented.

CHAPTER 4: IMPLEMENTATION

4.1 Introduction

The project was implemented in Python. The machine-learning pipeline lives in the `ml/` folder, the web backend in `backend/`, and the user interface in `frontend/`.

Purpose	Tools used
Data handling	pandas 2.3, NumPy 2.0
Machine learning	scikit-learn 1.6, XGBoost 2.1
Charts	matplotlib 3.9
Model storage	joblib 1.5
Web backend	Flask 3.1, Werkzeug, gunicorn
Database	SQLite
Frontend	HTML, CSS, JavaScript, Chart.js
Hosting and code	Render, GitHub

Table 4.1: Technology stack

4.2 Exploratory Data Analysis (EDA)

EDA was carried out on the 992 clean rows by `eda_figures()` in `ml/train.py`. Internal assessment turned out to be the strongest single signal of the final score (r = 0.84).

4.2.1 Distribution of Final Scores

```
fig, ax = plt.subplots(figsize=(7, 4))
ax.hist(df['final_score'], bins=25)
ax.set(title='Distribution of final scores', xlabel='Final score', ylabel='Students')
```

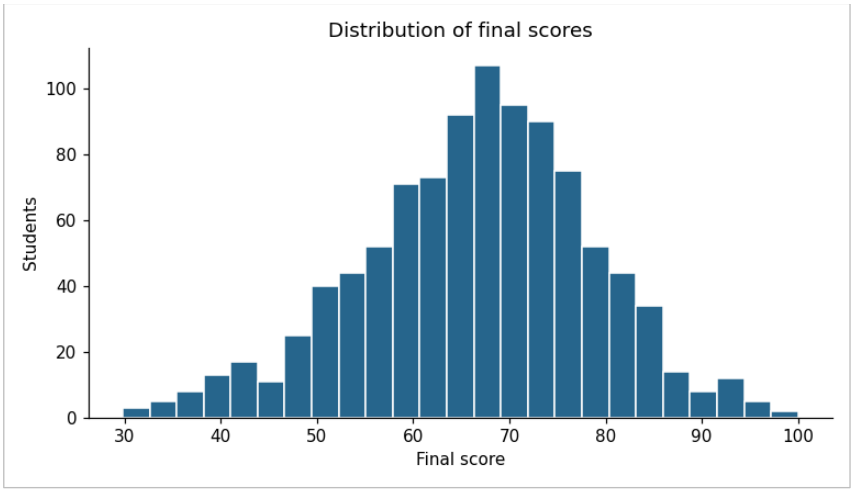


Figure 4.1: Histogram of final scores

Observation: scores average 66.6 and run from 29.8 to 100, roughly bell-shaped with a slightly longer low tail. Most students sit between 55 and 80, so the model has to separate the middle well, not just the extremes.

4.2.2 Feature Spread (Boxplots)

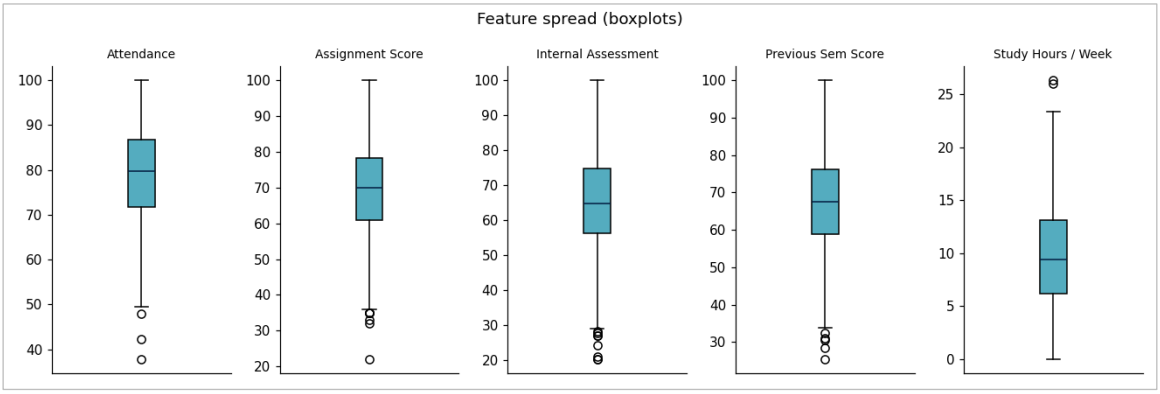


Figure 4.2: Boxplots of the five academic features

Observation: the middle half of students attend 72–87% of classes (median 80%), with a few outliers below 50%. Study hours have a median of about 9.5 a week, with the middle half between 6 and 13.

4.2.3 Scatter Plot Analysis

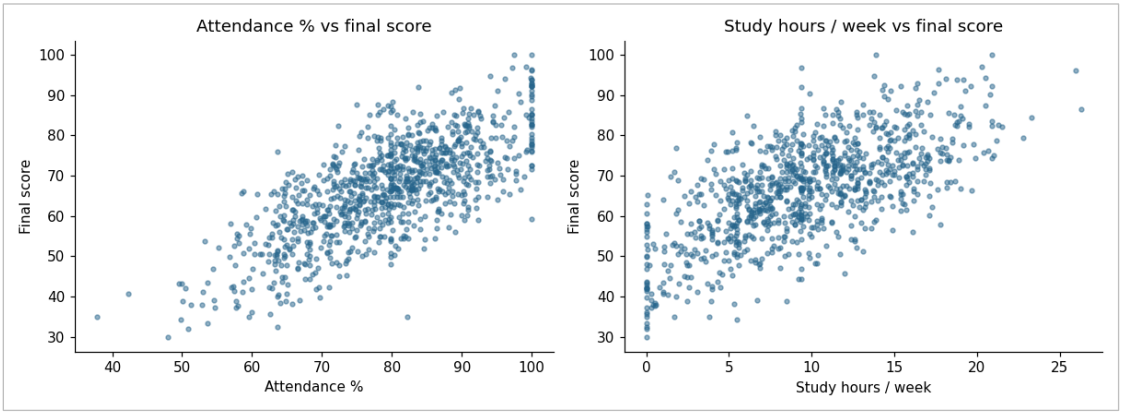


Figure 4.3: Attendance and study hours against the final score

Observation: both rise clearly with the score, and the wide vertical spread shows that no single feature predicts the score on its own. Beyond about 15 hours a week, extra study adds less (diminishing returns).

4.2.4 Correlation Analysis

```
corr = df.drop(columns=['student_id']).corr()
ax.imshow(corr, cmap='RdBu_r', vmin=-1, vmax=1)
```

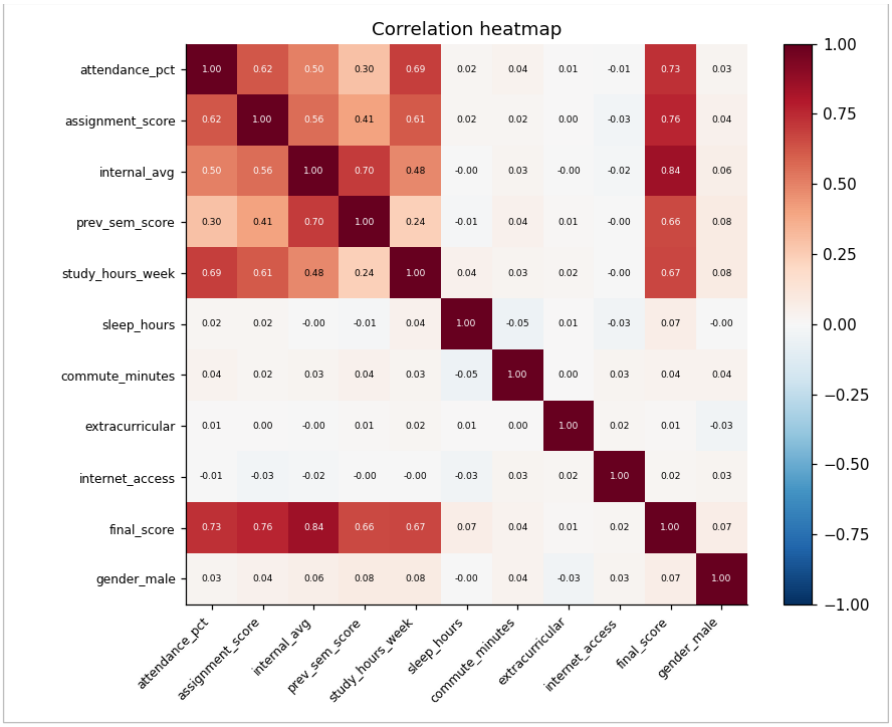


Figure 4.4: Correlation heatmap of all candidate features

Key findings: the five academic features correlate strongly with the final score and with each other. Sleep, commute, gender, extracurricular activity and internet access sit near zero (all $|r| < 0.08$).

4.3 Feature Engineering and Selection

Features were kept if their correlation with the final score was at least $|r| = 0.15$. Two engineered features were also built and kept only if they improved 5-fold cross-validated R^2 by more than 0.005.

4.3.1 Engineered Features

```
engagement_index = (df['attendance_pct'] + df['assignment_score']) / 2
study_efficiency = df['internal_avg'] / (df['study_hours_week'] + 1)
```

Engineered feature	Formula	r with score	CV R^2 change	Decision
engagement_index	(attendance + assignment) / 2	0.83	−0.0065	Dropped (repeats its parts)
study_efficiency	internal ÷ (study hours + 1)	−0.38	−0.0033	Dropped (no gain)

Table 4.2: Engineered features tested

4.3.2 Feature Selection Result

Feature	Correlation (r)	Decision
attendance_pct	0.731	Kept
assignment_score	0.759	Kept
internal_avg	0.844	Kept
prev_sem_score	0.663	Kept
study_hours_week	0.666	Kept
sleep_hours	0.070	Dropped (|r| < 0.15)
commute_minutes	0.035	Dropped (|r| < 0.15)
extracurricular	0.010	Dropped (|r| < 0.15)
internet_access	0.016	Dropped (|r| < 0.15)
gender_male	0.067	Dropped (|r| < 0.15)

Table 4.3: Feature selection by correlation with the final score

Selection kept exactly the five academic features that the application collects. The weakest kept feature (r = 0.66) is about ten times stronger than the strongest dropped one (r = 0.07).

4.4 Model Development

All three models were trained on the same 793 training rows with the five selected features.

```
models = {
  'Linear Regression': make_pipeline(StandardScaler(), LinearRegression()),
  'Random Forest': RandomForestRegressor(n_estimators=300, min_samples_leaf=3,
random_state=42),
  'XGBoost': XGBRegressor(n_estimators=400, learning_rate=0.05, max_depth=4,
subsample=0.9,
                          colsample_bytree=0.9, monotone_constraints='(1,1,1,1,1)'),
}
for name, model in models.items():
    cv_r2 = cross_val_score(model, X_train, y_train, cv=5, scoring='r2')
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
```

Model	Type	Key settings
Linear Regression	Linear model	StandardScaler + ordinary least squares
Random Forest	Bagging ensemble	300 trees, min 3 samples per leaf
XGBoost	Gradient boosting	400 trees, learning rate 0.05, depth 4, subsample 0.9, monotone constraints

Table 4.4: Model settings

The monotone constraints stop XGBoost from learning, for example, that more attendance lowers a score, which keeps its predictions sensible for the What-If simulator. The best model

is saved with joblib to `models/best_model.joblib`, and all metrics are written to `models/metrics.json`, which the application reads.

4.5 System Design and Architecture

The whole application runs as one Flask service. It serves the web pages, stores data in SQLite and loads the trained model once at startup. The browser never talks to the database or the model directly: every request goes through the API, which checks who is asking first.

Layer	Component	Responsibility
Frontend	HTML, CSS, JavaScript	Teacher portal, student portal, Drix assistant, charts
Backend	Flask API (backend/app.py)	Sign-in, students, attendance, notices, goals, predictions
Database	SQLite (6 tables)	Teachers, students, sessions, notices, notices read, attendance
Model	best_model.joblib	Predicts scores and factor impacts
Training	ml/train.py (offline)	Cleans data, trains 3 models, saves the best

Table 4.5: System components

Main API endpoints

Method	Endpoint	Purpose
POST	/api/auth/login	Sign in and receive a token
GET	/api/bootstrap	Load everything a portal needs
POST	/api/predict	Score, grade, risk and factor impacts
POST	/api/goal-plan	Ways for a student to reach a target score
GET/POST/PUT/DELETE	/api/students	Manage student records (teacher)
POST	/api/attendance	Save the attendance register (teacher)
POST/DELETE	/api/notices	Post and delete announcements (teacher)

Table 4.6: Main API endpoints

Security measures

- Passwords are stored only as salted hashes, never as plain text.
- Sign-in issues a random token that expires after 7 days.
- Each endpoint checks the caller's role; students receive only their own record plus class averages.
- All input is validated on the server, and repeated failed sign-ins are rate-limited.

4.6 Application Features

The application has two portals behind one sign-in page: teachers manage everything, and students see their own forecast and what to do about it.

Teacher portal

- **Dashboard:** class statistics, subject averages, grade distribution, top performers and at-risk students.
- **Manage Students:** add, edit and delete students; CSV import and export; report cards.
- **Attendance:** mark sessions present or absent; attendance and predictions update automatically.
- **Predict:** move sliders for the five inputs and see the predicted score and factor impacts live.
- **Model Lab:** compare the three trained models and view the training charts.
- **EDA & Analytics** and **Announcements**.



Figure 4.5: Teacher dashboard

Student portal

- **Overview:** predicted score, grade, risk level and class rank.
- **Performance:** subject strengths and how each factor moves the score.
- **What-If simulator, Goals and Tips:** plans to improve, with predicted gains.
- **Notices and Profile:** announcements, report card and password change.
- **Drix assistant:** a chatbot that answers questions from the student's own data.

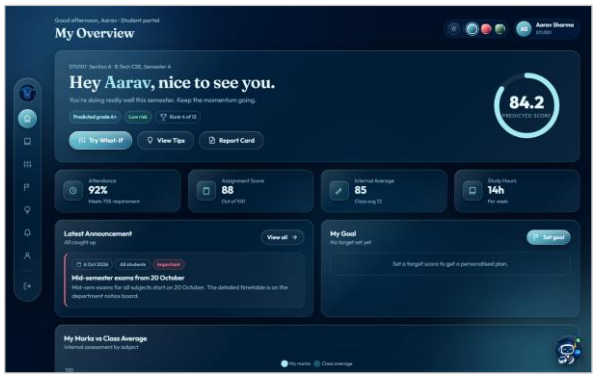


Figure 4.6: Student overview



Figure 4.7: Goal planner, target of 92

4.7 Deployment

The application is deployed on Render and is live at acadrix-da7i.onrender.com. The source code, trained model and dataset are on GitHub at github.com/Garima1223/acadrix-student-performance. Render builds the app from the repository using `render.yaml` and redeploys automatically on every push. To run it locally:

```
pip install -r requirements.txt
python backend/app.py
```

4.8 Chapter Summary

This chapter presented the EDA, feature engineering, model development, system architecture, application features and deployment. The next chapter presents and discusses the results.

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Introduction

This chapter presents the results of the three models on the 199 held-out test students and discusses why the deployed model performed best.

5.2 Model Performance Results

Model	MAE	MSE	RMSE	R^2 Score	5-fold CV R^2
Linear Regression (deployed)	3.32	17.38	4.17	0.901	0.893 ± 0.009
Random Forest	3.24	18.17	4.26	0.897	0.879 ± 0.009
XGBoost	3.33	18.28	4.28	0.896	0.888 ± 0.010

Table 5.1: Results on the test set



Figure 5.1: MAE and RMSE for the three models (lower is better)

Observation: Linear Regression achieved the best RMSE (4.17), R^2 (0.901) and cross-validated R^2 (0.893). Random Forest had the lowest MAE (3.24) but larger occasional errors. All three models landed within 0.006 R^2 of each other.

5.3 Actual vs Predicted Values

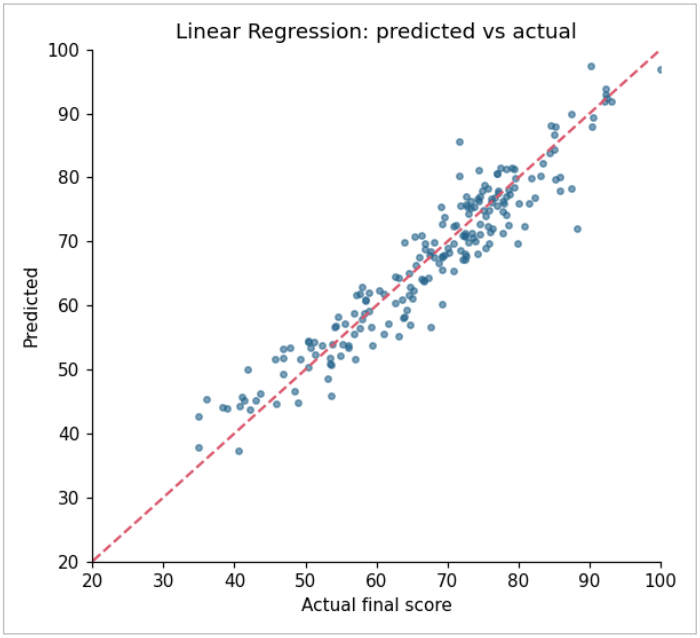


Figure 5.2: Predicted versus actual final scores (Linear Regression)

Observation: points follow the diagonal across most of the range. The exception is the lowest scorers: students who actually scored below about 45 are mostly predicted a few marks too high. The model therefore understates how badly the weakest students are doing, so a High-risk flag should be read as "at least this serious".

5.4 Feature Importance

Feature	Share of model weight
Internal Assessment	34.7%
Attendance	20.9%
Assignment Score	19.4%
Previous Sem Score	15.7%
Study Hours / Week	9.4%

Table 5.2: Feature importance in the deployed Linear Regression model

Internal assessment carries about a third of the model's weight. Study hours add the least once the other four are known, because diligent students also attend and score well.

5.5 Comparative Analysis

5.5.1 Why Linear Regression Won

The final score is mostly a weighted sum of the five features plus random noise (standard deviation 3.5 marks). The tree models can capture curves, but there is little curve to find, and they pay for their flexibility with slightly more variance. The remaining error is mostly the noise itself, which no model can learn. Linear Regression is also the fastest and the easiest to explain to a teacher.

5.5.2 Comparison with Published Studies

Finding in this project	What the literature reports	Agreement
Internal assessment is the strongest predictor (r = 0.84)	Internal assessment and CGPA most used [5]; prior achievement strongest [6]	Agrees
Previous semester score carries 16% of the weight	Earlier grades make the final grade easier to predict [2]	Agrees
Attendance correlates at r = 0.73	Meta-analysis: r = 0.44 with course grades [4]	Same direction; stronger in synthetic data
Lifestyle columns add nothing (|r| < 0.08)	Demographics weaker than academic history [6]	Broadly agrees
A linear model matches the ensembles	Ensembles and neural networks often lead [6]	Differs (near-linear data)

Table 5.3: Results compared with the literature

Most studies in Chapter 2 report classification accuracy, while this project reports regression error on an exact score, so the figures cannot be compared directly. The fair conclusion is that the method reproduces the patterns the literature expects; the absolute numbers need confirming on real student records.

5.6 Chapter Summary

Linear Regression achieved the best overall performance (R^2 0.901, RMSE 4.17, MAE 3.32) and was deployed. The next chapter presents the conclusion and future scope.

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Introduction

This chapter summarises the project, states its conclusions and limitations, and outlines future enhancements.

6.2 Project Summary

Acadrix was developed following the complete machine-learning lifecycle:

1. Phase 1: Problem understanding
2. Phase 2: Data collection
3. Phase 3: Data preprocessing
4. Phase 4: Exploratory data analysis
5. Phase 5: Feature engineering and selection
6. Phase 6: Model development
7. Phase 7: Model evaluation
8. Phase 8: Deployment

6.3 Conclusion

Acadrix predicts final scores to within about ±3.3 marks (R^2 = 0.901) from five indicators a college already records. The main lesson from the modelling is that a more complex model is not automatically better: Random Forest and XGBoost matched Linear Regression but did not beat it, because the relationship in the data is close to linear. The simplest model was the right one to deploy.

The value comes from turning predictions into action. Teachers see at-risk students weeks before exams, and students get specific, measurable steps (for example, "raise assignment scores to 85") instead of a bare grade.

6.4 Limitations

Limitation	Effect
Synthetic dataset	Shows the method works, not how real students behave
Lowest scorers predicted too high	High-risk students may look slightly better off than they are
Free hosting tier	App sleeps after 15 idle minutes; data resets on restart
Public demo accounts	Anyone with the link can sign in as the demo teacher
Rule-based assistant	Drix understands only the questions it was built for

Table 6.1: Limitations of the current system

6.5 Future Scope

Real Institutional Data

Retrain the model on real student records with the same five fields.

Better Predictions for Weak Students

Add more low-scoring examples or a model weighted toward that range.

Persistent Cloud Storage

Move to a paid plan with a persistent disk, or a hosted PostgreSQL database.

Integration with College ERP Systems

Import attendance and marks automatically instead of entering them.

Smarter Assistant

Connect Drix to a language model so it can answer free-form questions.

More Features

Collect and test factors such as health, family situation and teaching quality.

Explainable AI

Add per-student explanations such as SHAP values for tree-based models.

6.6 Final Conclusion

Acadrix demonstrates that machine learning can support educational institutions by identifying struggling students early and guiding them with personal plans. The project followed the complete pipeline from data to a deployed, secure web application, and with real data and permanent hosting it could be used by a department as it stands.

REFERENCES

Research Papers

[1] Romero, C., & Ventura, S. (2010). Educational data mining: A review of the state of the art. IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews), 40(6), 601–618.

[2] Cortez, P., & Silva, A. M. G. (2008). Using data mining to predict secondary school student performance. Proceedings of the 5th Future Business Technology Conference (FUBUTEC 2008), 5–12, Porto, Portugal. Dataset: UCI Machine Learning Repository, Student Performance.

[3] Kotsiantis, S., Pierrakeas, C., & Pintelas, P. (2004). Predicting students' performance in distance learning using machine learning techniques. Applied Artificial Intelligence, 18(5), 411–426. doi:10.1080/08839510490442058

[4] Credé, M., Roch, S. G., & Kieszczynka, U. M. (2010). Class attendance in college: A meta-analytic review of the relationship of class attendance with grades and student characteristics. Review of Educational Research, 80(2), 272–295.

[5] Shahiri, A. M., Husain, W., & Rashid, N. A. (2015). A review on predicting student's performance using data mining techniques. Procedia Computer Science, 72, 414–422. doi:10.1016/j.procs.2015.12.157

[6] Alyahyan, E., & Düştegör, D. (2020). Predicting academic success in higher education: Literature review and best practices. International Journal of Educational Technology in Higher Education, 17, 3.

[7] Albreiki, B., Zaki, N., & Alashwal, H. (2021). A systematic literature review of student' performance prediction using machine learning techniques. Education Sciences, 11(9), 552.

[8] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32.

[9] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD 2016). arXiv:1603.02754

[10] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Websites and Documentation

[11] scikit-learn documentation — https://scikit-learn.org

[12] pandas documentation — https://pandas.pydata.org

[13] XGBoost documentation — https://xgboost.readthedocs.io

[14] Flask documentation — https://flask.palletsprojects.com

[15] Render documentation — https://render.com/docs

APPENDIX

A. Project Links

- Source code: https://github.com/Garima1223/acadrix-student-performance
- Live application: https://acadrix-da7i.onrender.com

B. Demo Accounts

Role	Username	Password
Teacher	teacher	teacher@123
Student	STU101 to STU112	student@123

C. Project Structure

```
ml/
  features.py           feature list, grades and risk bands
  generate_dataset.py   seeded synthetic dataset
  train.py              preprocessing, EDA, feature selection, training, evaluation
  predictor.py          loads the model; predictions, factor impacts, goal plans
backend/
  app.py                Flask API, also serves the frontend
  database.py           SQLite schema and demo data
frontend/               index.html, app.js, styles.css, assets/
data/                   students_dataset.csv
models/                 best_model.joblib, metrics.json
reports/figures/        charts used in this report
render.yaml             Render deployment settings
```